RAG evaluation framework: RAGAS and the alternatives

Daily standup, Monday 26 October 2026 | FEDIS internship, Ticket-Easy production readiness | Engineer: Yousef | This week's task: *choose a RAG evaluation framework (e.g. RAGAS) and apply it to the current test dataset*

Standup in 30 seconds

- **Yesterday:** picked up the framework task. Read the knowledge service's evaluation code, the benchmark files and the 16-17 October eval report, and made a shortlist of frameworks.
- **Today:** write the adapter that turns our existing data into evaluation samples, and run a first RAGAS pass on shop_001 (37 test questions plus the logged conversation replies).
- **Blocker / decision needed:** which model judges the replies. Every run so far used free tiers. A judge doing several calls per sample will hit free-tier rate limits, and it has to read Egyptian Arabic and Arabizi.
- **Proposal:** keep our own exact metrics, and add RAGAS for what we cannot measure today: is the reply true to the passage, and does it answer the question. DeepEval is the runner-up.

1. Why this task matters for Ticket-Easy

We already measure retrieval well, but only one thing: is the expected policy section in the top 5. The open gaps in the eval report are about what happens *after* retrieval, and the current numbers cannot explain them:

Gap in the 17 Oct report	Now	Target	What we can't answer today
Reply cites the right policy passage	70.7%	95%	Is the reply wrong, or right but citing a different section? Is the passage noisy?
Decision correct	81.7%	92%	Do policy turns fail at retrieval, at the answer, or because of a clarify/handover?
No-answer recall (unanswerable questions)	33-78%	-	When off-topic passages come back, does the reply still use them (a hallucination risk)?
telco_001 recall@5 (test)	82.22%	-	Which chunks are irrelevant, and are they ranked above the right one?

The point of the work: a framework is only worth adding if it tells us *where* a policy answer goes wrong (retrieval, ranking or generation), so the team can fix the right part. Adding a dashboard of scores nobody acts on is not the point.

2. What the current dataset gives us

Source in the repo	What a record holds	Size	Usable for
rag/data/*/benchmark/retrieval.{dev, test}.jsonl	question, style (en / ar / arabizi), expected section ids; empty list = unanswerable	403 questions (203 test); 45 unanswerable in test	Retrieval metrics with no LLM: ID-based context precision/recall, MRR, nDCG
Docs/output/eval_runs/diagnosis/*.jsonl	customer message, bot reply, cited section ids, gold citations, intent, decision	about 2,000 logged turns	Generation metrics: faithfulness, answer relevancy, context precision (needs passage text from the corpus)
rag/data/*/corpus	the policy documents behind each section id	3 businesses	Passage text for the judge
shop_001/benchmark/guardrails.jsonl	tool request + facts + expected allow/deny	44 cases	Not a RAG metric. Keep it in our own suite.

What's missing: **reference answers** (a gold reply per question). Metrics like answer correctness and LLM-based context recall need them. The plan below writes them for a small calibration set only, rather than for all 400+ questions.

3. What a framework must handle to fit Ticket-Easy

- **Language:** English, Egyptian Arabic, mixed Arabic-English and Arabizi. LLM judges are mostly checked in English, so a judge score on Arabizi is not trustworthy until we measure it.

- **Cost:** no model call so far was paid for. The judge runs on Groq / OpenRouter free tiers, with retries and caching, unless a budget is approved.
- **Gold data are section ids, not answers.** Prefer metrics that use ids directly. They are exact, free and repeatable.
- **CI:** a weekly workflow (.github/workflows/eval.yml) already runs evals. The new metrics should run there and be compared against a committed baseline, like the current ones.
- **Data stays local:** customer conversations should not go to a hosted evaluation platform. This also ties in with this week's PII task.

4. Options compared

	RAGAS	DeepEval	TruLens	Arize Phoenix	promptfoo	Our own harness (today)
What it is	RAG metric library	pytest-style LLM test framework	RAG triad + feedback functions	Tracing + evals UI	YAML test runner, red-teaming	recall@5, no-answer P/R, citation checks
Retrieval metrics without an LLM	Yes (ID-based precision/recall, non-LLM string metrics)	Mostly LLM-judged	LLM-judged	Some	Basic	Yes, exact
Faithfulness / grounding	Yes	Yes	Yes, per chunk	Yes	Yes (via rubric)	Partly ('unbacked facts' check)
Works with our gold section ids	Directly	Needs mapping	Needs mapping	Needs mapping	Custom asserts	Directly
CI gating	Script + threshold	Built in (asserts fail the build)	Weak	Weak	Built in	Yes (eval-compare)
Custom judge model (Groq / OpenRouter)	Yes	Yes	Yes	Yes	Yes	n/a
Arabic / Arabizi	As good as the judge; prompts can be adapted	As good as the judge	As good as the judge	As good as the judge	As good as the judge	Handled (style tags)
Licence	Apache-2.0	Apache-2.0	MIT	Elastic 2.0 (not OSI open source)	MIT	ours
Fit for this task	**Best fit**	Strong runner-up	Fine, overlaps RAGAS	Later, for production tracing	Better for guardrail red-teaming	Keep as the exact layer

Sources: framework docs plus 2026 comparison write-ups (Analytics Vidhya, Atlan, FutureAGI). Several comparisons come from vendors, so treat their rankings as directional. RAGAS class names changed in the 0.4 line (new *metrics.collections* API), so exact names get checked against the installed version on day 1. One 2026 write-up reports slower RAGAS maintenance. This is a reason to pin the version and keep the adapter thin, so we can switch to DeepEval if needed.

5. Recommendation: two layers, not one tool

Layer	Tool	Metrics	Answers the question
Exact (every run, free)	Our harness + RAGAS ID-based metrics	recall@5, ID context precision/recall, MRR, no-answer P/R, per style	Did the right section come back, and how high was it ranked?
Judged (weekly, sampled)	RAGAS with a pinned free-tier judge	Faithfulness, response relevancy, LLM context precision	Is the reply true to the passage, and does it answer the customer? Is the 70.7% citation gap a retrieval or a generation problem?
Later (production)	Arize Phoenix or Langfuse (self-hosted)	Traces + sampled online evals	What is going wrong with real customers, not just the test set?

Why RAGAS over DeepEval: RAGAS can score retrieval straight from our section ids, it is focused on RAG, and it is the name in the task. DeepEval wins if Yousef wants hard pass/fail gates inside pytest. To keep the choice honest, both run on the same 30-sample calibration set and the one whose scores agree more with human labels is

the one we keep.

6. The main risk, and how we check it

An LLM judge can score an Arabizi reply badly just because it reads Arabizi badly. Before any judged score goes into a report:

- Hand-label 30 replies from the diagnosis logs (10 en / 10 ar / 10 arabizi). Mark each one faithful or not, and answers the question or not, and write a short reference answer.
- Run the judge on the same 30 and measure agreement with the human labels, per language style.
- Only report a judged metric for a style where agreement is at least 80%. Otherwise report it as 'not reliable yet', and try a stronger judge or a translated-input check.

7. Plan for the week

Day	Work	Done when
Mon (today)	Adapter: retrieval benchmark + diagnosis logs + corpus text into RAGAS samples. Pin the RAGAS version.	A samples file for shop_001; ID-based metrics match our recall@5
Tue	First judged run on shop_001 with the free-tier judge, with caching and rate-limit retries.	Faithfulness / relevancy per style; token usage logged like the current usage/*.jsonl
Wed	Calibration: 30 hand labels, run RAGAS vs DeepEval on the same set.	Agreement table per style, and a final pick of framework and judge
Thu	Full run on noon_eg + telco_001; add a make target and a step to the weekly CI workflow.	Committed baseline; eval-compare covers the new metrics
Fri	Write-up: what the new metrics say about the 70.7% citation gap and no-answer recall.	One-page findings with the next fix for the team

8. Questions for Yousef

- Can the judge use a small paid budget, or must it stay on free tiers? This decides how big the judged runs can be.
- Is a pytest-style pass/fail gate in CI wanted (this points to DeepEval), or are tracked scores enough (RAGAS)?
- Can the diagnosis logs be sent to an external judge API, or should those calls wait for the PII masking POC?